Vendor Technical Governance & Anti-Lock-In Playbook (Google + Systems Integrators)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Vendor Technical Governance & Anti-Lock-In Playbook (Google + Systems Integrators) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Vendor Technical Governance & Anti-Lock-In Playbook 5

Why this exists . . . 5

The operating model — where authority sits . . . 5

The artifacts this method names and reuses . . . 5

What "good" looks like . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Vendor Technical Governance & Anti-Lock-In Playbook 7

2.1 Named enterprise architectural authority . . . 7

2.2 Target Architecture Baseline . . . 7

2.3 Anti-Lock-In Criteria adopted . . . 8

2.4 Governance RACI . . . 8

2.5 Vendor design register (inventory source) . . . 8

2.6 Escalation path . . . 8

2.7 Microsoft Graph PowerShell SDK (for source-environment recon) . . . 8

2.8 A pilot design and change control . . . 9

Checklist . . . 9

Global Requirements — Google Workspace & Gemini Solutions 9

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 10

4. Identity & provisioning source of truth . . . 10

5. A pilot scope (OU / group) and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

3. Implementation Guide — Vendor Technical Governance & Anti-Lock-In Playbook 11

Step 1 — Stand up the vendor design register . . . 12

Step 2 — Establish the design-authority gate . . . 12

Step 3 — Adopt the Anti-Lock-In Criteria and RACI . . . 12

Step 4 — The vendor-design review method (per design) . . . 14
Step 5 — Pilot on a first design . . . 14
Step 6 — Handle non-conformant designs (escalation) . . . 14
Step 7 — Validate governance is holding . . . 15
Step 8 — Expand across all partners and workstreams . . . 15
5. Testing Guide — Vendor Technical Governance & Anti-Lock-In Playbook 15
Test 1 — Register is complete . . . 16
Test 2 — Every design is dispositioned . . . 16
Test 3 — Anti-lock-in criteria are scored . . . 16
Test 4 — A non-conformant design is caught, not passed . . . 16
Test 5 — Assumptions are validated before build . . . 16
Test 6 — No un-approved vendor object exists . . . 16
Test 7 — Findings report is accurate . . . 16
Test 8 — Exceptions are tracked . . . 16
Pass/fail gate . . . 17
6. Troubleshooting Guide — Vendor Technical Governance & Anti-Lock-In Playbook 17
A. A vendor design fails one or more anti-lock-in criteria . . . 17
B. A partner pushes back on the gate . . . 17
C. Escalation ladder for a rejected/blocked design . . . 18
D. Exceptions piling up / going stale . . . 18
E. A vendor object exists with no approved design . . . 18
F. Register out of date / discovery returns little . . . 19
G. Diagnostics quick reference . . . 19
H. When to escalate to program leadership . . . 19
7. Best Practices — Vendor Technical Governance & Anti-Lock-In Playbook 19
8. Deliverables Summary — Vendor Technical Governance & Anti-Lock-In Playbook 20
Documentation . . . 20
PowerShell scripts . . . 21
The Vendor-Governance Runbook (outline) . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Vendor Technical Governance & Anti-Lock-In Playbook (Google + Systems Integrators)

White-label · Google

On a large Google Workspace and Gemini transformation, the people doing the work are usually not your people. Google's delivery teams and one or more systems integrators drive the design, the migration, and the integration build. Left ungoverned, this quietly erodes architectural control: decisions get made inside vendor design documents, defaults get accepted under schedule pressure, and proprietary shortcuts get baked in because they are the fastest path for the party being paid to finish. The result is an estate the enterprise did not really design, cannot easily change, and cannot affordably leave.

This is a complete, deploy-it-today playbook that makes the enterprise architect the **primary technical counterpart** to those partners. It gives you a repeatable operating model to validate vendor assumptions before they harden, review vendor designs against your target architecture, apply an explicit anti-lock-in bar to every design, and keep a clean separation between **vendor execution responsibility** and **enterprise architectural authority** — so partners deliver sustainable solutions without fragmentation or lock-in.

What you get

• A plain-language Solution Overview of the operating model between the enterprise architect and

partners, where architectural authority sits, and the durable artifacts the method names and reuses (Target Architecture Baseline, Anti-Lock-In Criteria, Vendor Assumption Log, Design Review Record, Governance RACI, Vendor-Governance Runbook).

• A Requirements Guide with verify / implement-if-missing for every prerequisite.

• An Implementation Guide containing the vendor-design review method, the four-part **Anti-Lock-In

Criteria **checklist (portability, data ownership, standards conformance, exit criteria), the** Governance RACI** that separates execution from authority, and the escalation path — in ordered steps.

• Five PowerShell scripts for design-register discovery, anti-lock-in conformance validation, a

review/finding report, remediation-action recording, and a reviewer/pilot scope builder.

• Testing and Troubleshooting guides — governance and conformance gates, and how to handle

non-conformant designs.

What this delivers

A governed partner delivery model — every vendor design reviewed against a fixed anti-lock-in bar and dispositioned by the enterprise, assumptions validated before they become constraints, non-conformant designs escalated rather than waved through, and a clean, signed RACI keeping execution and architectural authority separate — proven on a pilot design before it gates every workstream.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Vendor execution that aligns to the target architecture — no long-term lock-in, no fragmentation — with architectural control firmly held by the enterprise.

1. Solution Overview — Vendor Technical Governance & Anti-Lock-In Playbook

Why this exists

On a large Google Workspace and Gemini transformation, the people doing the work are usually not you. Google's own delivery teams and one or more systems integrators drive the design, the migration, and the integration build. That is fine — it is what they are good at — but it creates a quiet, structural risk: the enterprise can lose architectural control. Decisions get made inside vendor design documents, defaults get accepted because challenging them slows the schedule, and proprietary shortcuts get baked in because they are the fastest path for the party being paid to finish. Six months later the enterprise owns an estate it did not really design, cannot easily change, and cannot leave.

This playbook makes the enterprise architect the **primary technical counterpart** to those partners. It is a repeatable operating model: validate vendor assumptions before they harden, review vendor designs against the target architecture, apply an explicit anti-lock-in bar to every design, and keep a clean separation between **vendor execution responsibility** and **enterprise architectural authority**. Partners still deliver the work — but they deliver work that aligns to the architecture the enterprise owns.

The operating model — where authority sits

The relationship has two distinct roles, and the whole method depends on not blurring them.

- **Vendor execution responsibility.** Google and the systems integrators are responsible for

how the work is built and delivered — the migration runbooks, the integration code, the configuration effort, the schedule. They are accountable for delivering to specification, on time, and to quality.

- **Enterprise architectural authority.** The enterprise architect is responsible for *what the

target architecture is and whether a proposed design conforms to it* — the standards a design must meet, the anti-lock-in criteria it must satisfy, and the sign-off that lets a design proceed to build. This authority is **not** delegable to a vendor. A vendor may propose; the enterprise disposes.

The mechanism that keeps this clean is a **design-authority gate**: no vendor design proceeds to build until it has been reviewed against the target architecture and the anti-lock-in criteria and has been dispositioned (Approved / Approved-with-conditions / Rejected) by the enterprise. The gate is the point where architectural authority is actually exercised, not just asserted.

The artifacts this method names and reuses

The playbook works because it names a small set of durable artifacts and uses them on every design:

- **Target Architecture Baseline** — the enterprise's own statement of the intended end state

(identity model, data ownership, integration patterns, standards, exit posture). Every vendor design is judged against this, not against the vendor's convenience.

• **Anti-Lock-In Criteria** — the fixed checklist every design must pass: **portability**, **data ownership**, **standards conformance**, and **exit criteria** (see §3). These are the non-negotiable bar.

• **Vendor Assumption Log** — the running list of assumptions a vendor design depends on, each marked validated / unvalidated / rejected, so an unchallenged assumption cannot silently become a design constraint.

• **Design Review Record** — the per-design finding sheet: what was reviewed, which criteria passed or failed, the disposition, and the conditions attached.

• **Governance RACI** — the one-page map of who is Responsible, Accountable, Consulted, and Informed for each decision type, drawing the execution-vs-authority line explicitly (see §3).

• **Vendor-Governance Runbook** — the repeatable procedure the enterprise runs for each incoming design (see §8).

What "good" looks like

• Every vendor design is reviewed against the same criteria, by the same method, with a recorded disposition — not approved by schedule pressure or by whoever was in the room.

• Assumptions are surfaced and validated *before* they harden into the build.

• Lock-in is caught at design time, when it is cheap to change, instead of discovered at exit, when it is expensive or impossible.

• The enterprise can always answer three questions about any part of the estate: *who owns the data, how would we move it, and what standard does it conform to.*

Benefits

• **Held architectural control** — authority stays with the enterprise; vendors execute against a design the enterprise owns and signed off.

• **Anti-lock-in by default** — portability, data ownership, standards, and exit are checked on every design, so proprietary dependencies do not accumulate unnoticed.

• **Fewer surprises** — validated assumptions and reviewed designs mean fewer late reversals and less rework.

• **Durable partner relationships** — a clear, consistent bar makes the enterprise a better client: vendors know exactly what "good" is and can deliver to it.

• **Repeatability** — a governance method you run on every design and every partner, not a one-off architecture review.

Limitations (be honest with buyers)

• This is a **governance and review method**, not a replacement for the vendor's delivery

capability or for a full architecture practice. It makes vendor work conform; it does not do the vendor's work.

• Applying the anti-lock-in bar can slow an individual design or require a partner to rework a

proposal — that friction is the point, but it is real and must be managed through the escalation path (§3), not by waiving the gate.

• The included scripts inventory and validate the *artifacts* of vendor delivery (design register,

and vendor-created objects readable in the Microsoft 365 / Entra source environment); they do not audit the internal correctness of a partner's proprietary build — that stays a human review against the criteria.

• Anti-lock-in reduces switching cost; it does not make any enterprise platform costlessly

portable. Some coupling is a deliberate, accepted trade — the method makes that trade *explicit and owned* rather than accidental.

The standard deployment model

> *Assess the current partner operating model and target architecture -> Configure the governance gate, RACI, and anti-lock-in criteria -> Pilot the review method on a first vendor design -> Validate that designs are dispositioned and conform -> Expand the gate across all partners and workstreams. Repeatable pattern:* ***Assess -> Configure -> Pilot -> Validate -> Expand.***

2. Requirements Guide — Vendor Technical Governance & Anti-Lock-In Playbook

Verify every requirement before running the governance gate. Each entry: Description / Why / Verify / Implement-if-missing. Category-wide prerequisites are in `global-requirements-google.md`.

2.1 Named enterprise architectural authority

Description: A named enterprise architect (or a small architecture-authority group) empowered to disposition vendor designs — Approve, Approve-with-conditions, or Reject — and whose sign-off is a required gate before a design proceeds to build. Why: The whole method rests on architectural authority being held by the enterprise, not delegated to a vendor. Without a named authority with real sign-off power, the gate is advisory and gets overridden by schedule pressure. Verify: Confirm the role exists, is named in the program governance, and its sign-off is a documented precondition for build. Implement: Name the authority in the program charter; make the design-authority gate a mandatory step in the delivery process (see Implementation Step 2).

2.2 Target Architecture Baseline

Description: The enterprise's own written statement of the intended end state — identity model, data-ownership position, sanctioned integration patterns, the standards designs must conform to, and the

required exit posture. Why: Every vendor design is reviewed *against this baseline*. Without it, reviews default to the vendor's assumptions and there is nothing to hold a design to. Verify: A Target Architecture Baseline document exists and is version-controlled; it states the identity, data-ownership, integration, standards, and exit positions. Implement: Author the baseline (the Google Workspace / Gemini target architecture blueprint is the natural source); if none exists, capture at least the four anti-lock-in positions (§3) before the first review.

2.3 Anti-Lock-In Criteria adopted

Description: The fixed, four-part bar every design must pass — **portability**, **data ownership**, **standards conformance**, and **exit criteria** — adopted as enterprise policy, not negotiated per design. Why: A consistent bar applied to every design is what stops lock-in accumulating; a bar that is re-argued each time is no bar at all. Verify: The criteria are documented and referenced by the review process; `GWVEND-Validation.ps1` scores a design register against them. Implement: Adopt the criteria in §3 as-is; tailor thresholds to the enterprise's risk posture and publish them.

2.4 Governance RACI

Description: A one-page RACI that assigns Responsible / Accountable / Consulted / Informed for each decision type, drawing the line between **vendor execution responsibility** and **enterprise architectural authority**. Why: The execution-vs-authority separation only holds if it is written down; ambiguity is where authority quietly migrates to whoever is doing the work. Verify: A RACI exists, is agreed with the partners, and names the enterprise as Accountable for architectural disposition and the vendors as Responsible for execution. Implement: Adopt the RACI in §3; review and sign it with each partner at engagement start.

2.5 Vendor design register (inventory source)

Description: A single register of the vendor deliverables and designs in flight — design documents, proposed integration patterns, migration approaches — each with its owner (which partner), its status, and its review disposition. Maintained as a structured file (CSV/JSON) the scripts can read. Why: You cannot govern designs you have not inventoried; on a multi-partner program, designs arrive faster than any one person tracks by memory. Verify: Run `GWVEND-Discovery.ps1`; confirm it enumerates the register and any vendor-created objects it can read in the source environment, with owner and status. Implement: Stand up the register (see Implementation Step 1); seed it from the current in-flight vendor deliverables.

2.6 Escalation path

Description: A defined path for handling a design the enterprise cannot approve — how a rejected or conditioned design is escalated, who arbitrates a partner disagreement, and how a time-boxed exception is granted and tracked. Why: A gate with no escalation path becomes a bottleneck that gets bypassed. The path is what keeps the gate authoritative *and* the program moving. Verify: The escalation and exception process is documented and agreed with the partners. Implement: Adopt the escalation ladder in §3 and record it in the program governance.

2.7 Microsoft Graph PowerShell SDK (for source-environment recon)

Description: The Microsoft Graph PowerShell SDK on the operator workstation, with consent to the read scopes the scripts use, for the coexistence/migration solutions where vendors create objects in the source

Microsoft 365 / Entra environment. Why: Discovery and validation read vendor-created enterprise apps, groups, and configuration in the source tenant to cross-check the design register against what has actually been built. Verify: `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; a test `Connect-MgGraph` succeeds. Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`; the scripts request the least privilege they need (`Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`, and — only for the scope helper's create mode — `Group.ReadWrite.All`).

2.8 A pilot design and change control

Description: A first, representative vendor design to run the review method on, and a change-control process so the governance gate is inserted into production delivery under control. Why: Every rollout follows Assess -> Configure -> Pilot -> Validate -> Expand; the pilot proves the method on one design before it gates every workstream. Verify: A named pilot design exists and a change/approval path is available. Implement: Pick a representative in-flight design; agree the change-control insertion of the gate with the program (see Implementation Step 3).

Checklist

- Named enterprise architectural authority with real sign-off power
- Target Architecture Baseline authored and version-controlled
- Anti-Lock-In Criteria (portability, data ownership, standards, exit) adopted as policy
- Governance RACI agreed and signed with each partner
- Vendor design register stood up — `GWVEND-Discovery.ps1`
- Escalation and exception path documented and agreed
- Microsoft Graph PowerShell SDK installed and consented (for source-env recon)
- Pilot design selected and change-control insertion agreed

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Vendor Technical Governance & Anti-Lock-In Playbook

Follow these steps in order. The method is run by the enterprise architect; the scripts support the inventory, validation, and reporting steps. Console references are to the Microsoft Entra admin center (`https://entra.microsoft.com`) and the Google Admin console (`https://admin.google.com`) where the source-environment recon touches them.

> *Model: establish the register and the gate, adopt the anti-lock-in criteria and RACI, pilot the review method on one vendor design, validate that designs are dispositioned and conform, then expand the gate across every partner and workstream. Every design proceeds only by disposition.*

Step 1 — Stand up the vendor design register

1. Create a structured register (CSV or JSON) of every vendor deliverable and design in flight. Per entry record: `DesignId`, `Title`, `Owner` (which partner — keep it generic, e.g. "Integrator A" / "Platform vendor"), `Type` (integration pattern / migration approach / configuration design), `Status` (proposed / in-review / approved / rejected), and the four anti-lock-in fields the review will fill (`Portability`, `DataOwnership`, `StandardsConformance`, `ExitCriteria`).

1. Seed it from the current in-flight deliverables.

2. Run `GWVEND-Discovery.ps1` to enumerate the register and cross-check it against any vendor-created objects readable in the source Microsoft 365 / Entra environment. Save the output — it is your governance baseline.

Step 2 — Establish the design-authority gate

1. Insert a mandatory gate into the delivery process: no vendor design proceeds to build until the enterprise architect has dispositioned it (Approved / Approved-with-conditions / Rejected).

1. Define the disposition record (the **Design Review Record**): design reviewed, criteria results, disposition, conditions, reviewer, date.

1. Communicate to every partner that the gate is a precondition for build, not an optional review.

Step 3 — Adopt the Anti-Lock-In Criteria and RACI

1. Adopt the four-part **Anti-Lock-In Criteria** (below) as enterprise policy.
2. Adopt and sign the **Governance RACI** (below) with each partner at engagement start.
3. Publish both alongside the Target Architecture Baseline so partners design *to* the bar.

The Anti-Lock-In Criteria (the fixed bar every design must pass)

#	Criterion	What the design must demonstrate
1	**Portability**	Workloads, configuration, and integrations can be moved or rebuilt on an alternative without a proprietary rewrite; sanctioned/standard patterns are used over vendor-only shortcuts; no hard dependency on a single-vendor-only feature without a documented, accepted reason.

#	Criterion	What the design must demonstrate
2	**Data ownership**	The enterprise owns the data and the schema; data is stored in open or documented formats; the enterprise can extract a complete, usable copy on demand — not just a vendor-gated export.
3	**Standards conformance**	Identity, integration, and data-exchange use open standards (e.g. SAML/OIDC/SCIM, documented APIs, standard file/data formats) rather than proprietary equivalents where a standard exists.
4	**Exit criteria**	A credible exit exists and is documented: how the workload would be migrated off, what the switching cost is, what data comes with it, and what contractual/technical dependencies would have to be unwound. A design with no articulable exit fails.

A design that fails any criterion is **Rejected** or **Approved-with-conditions** (with the failing criterion as the condition). "We'll deal with exit later" is a fail, not a pass.

The Governance RACI (execution vs architectural authority)

Decision / activity	Enterprise Architect	Program / Enterprise Owner	Vendor / Integrator
Define the Target Architecture Baseline	**A/R**	C	I
Set the Anti-Lock-In Criteria	**A/R**	C	I
Produce a vendor design	C	I	**A/R**
Validate assumptions behind a design	**A/R**	C	R
Disposition a design (gate sign-off)	**A/R**	C	I
Build/deliver an approved design	I	A	**R**
Grant a time-boxed exception	C	**A**	R

Decision / activity	Enterprise Architect	Program / Enterprise Owner	Vendor / Integrator
Escalate a rejected/blocked design	R	**A**	R

Read the split plainly: **the enterprise architect is Accountable for architectural disposition; the vendor is Responsible for execution.** Authority never sits in the "produce a design" or "build" rows.

Step 4 — The vendor-design review method (per design)

Run this for each design as it arrives:

1. **Log assumptions.** Extract the assumptions the design depends on into the **Vendor Assumption Log**; mark each validated / unvalidated / rejected. Do not let an unvalidated assumption pass into review as fact.

1. **Review against the baseline.** Compare the design to the Target Architecture Baseline — identity, integration pattern, data ownership, standards.

1. **Apply the Anti-Lock-In Criteria.** Score the design on all four criteria. Use `GWVEND-Validation.ps1` to score the register entry consistently and flag any missing/failing criterion.

1. **Challenge.** Where the design uses a proprietary shortcut, require the sanctioned/standard pattern or a documented, accepted reason. Where it assumes a default, require the assumption be validated.

1. **Disposition.** Record Approved / Approved-with-conditions / Rejected in the Design Review Record, with conditions. Only an Approved (or conditions-met) design proceeds to build.

1. **Report.** Run `GWVEND-Report.ps1` to produce the review/finding report for the design or the whole register.

Step 5 — Pilot on a first design

1. Run the full Step 4 method on the pilot design.

2. Confirm the design is dispositioned, the criteria results are recorded, and any conditions are tracked to closure.

1. Confirm `GWVEND-Validation.ps1` and `GWVEND-Report.ps1` reflect the disposition. Complete the Testing Guide.

Step 6 — Handle non-conformant designs (escalation)

1. A **Rejected** or blocked design goes to the escalation ladder (see §6 for the detailed handling):

• Architect and vendor attempt resolution against the specific failing criterion.

• If unresolved, the Program/Enterprise Owner arbitrates (RACI: Accountable for escalation).

• A time-boxed **exception** may be granted (owner-accountable) — recorded, dated, and with a remediation commitment; it is not a silent waiver.

1. Track every exception to closure; an unclosed exception is standing lock-in debt.

Step 7 — Validate governance is holding

1. Run `GWVEND-Validation.ps1` -> confirms every register entry has a disposition, no build-status design is missing a passed criterion, and no exception is past its close date.

1. Confirm the source-environment recon shows no vendor-created object that has no corresponding approved design. Complete the Testing Guide gates.

Step 8 — Expand across all partners and workstreams

1. With the method proven on the pilot, make the gate mandatory for every partner and every workstream.

1. Put governance on a cadence: re-run `GWVEND-Discovery.ps1` and `GWVEND-Report.ps1` regularly so new designs and vendor-created objects are caught as they appear.

1. Run the **Vendor-Governance Runbook** (see §8) for each incoming design so the method is executed the same way regardless of which partner or architect is involved.

Order of operations

```
Register + gate -> criteria + RACI -> review method -> pilot -> escalation for non-conformance -> validate -> expand
```

Rollback / reversibility

• The gate and criteria are process controls — they add a review step and change nothing in a live system, so "rollback" is simply relaxing a condition or granting a tracked exception, never a technical revert.

• The only script that writes is the scope helper (`GWVEND-Set-ScopeTargeting.ps1`), which creates reviewer/pilot groups and defaults to preview (`-WhatIf`); it creates empty groups only, which are trivially removed if not needed.

5. Testing Guide — Vendor Technical Governance & Anti-Lock-In Playbook

Run after the pilot configuration (Implementation Step 5) and on the recurring governance cadence. These are **governance and conformance gates**: they test that the method is being applied and that designs actually meet the anti-lock-in bar — not that a live system is up.

Test 1 — Register is complete

Do: Run `GWVEND-Discovery.ps1`. Expected: Every in-flight vendor deliverable/design is listed with owner and status, and any vendor-created object readable in the source Microsoft 365 / Entra environment is enumerated and cross-checked against the register — a usable governance baseline.

Test 2 — Every design is dispositioned

Do: Run `GWVEND-Validation.ps1`. Expected: Every register entry has a disposition (proposed entries are in-review; no entry is at build status without an Approved / conditions-met disposition).

Test 3 — Anti-lock-in criteria are scored

Do: Run `GWVEND-Validation.ps1` and spot-check one flagged design against its Design Review Record. Expected: Each design shows a pass/fail on all four criteria (portability, data ownership, standards conformance, exit criteria); any design missing a criterion or failing one is flagged, and the flag matches the recorded review.

Test 4 — A non-conformant design is caught, not passed

Do: Enter (or use) a design that uses a proprietary-only pattern with no documented exit and run `GWVEND-Validation.ps1`. Expected: The design fails standards-conformance and/or exit-criteria and is reported as non-conformant — it is **not** silently marked conformant.

Test 5 — Assumptions are validated before build

Do: Review the Vendor Assumption Log for a design at build status. Expected: No unvalidated assumption remains for a design that has proceeded to build; every assumption is validated or rejected.

Test 6 — No un-approved vendor object exists

Do: Run `GWVEND-Discovery.ps1` and compare the vendor-created objects found in the source environment against the approved designs. Expected: Every vendor-created object maps to an approved design; any object with no corresponding approved design is flagged for review (built ahead of the gate).

Test 7 — Findings report is accurate

Do: Run `GWVEND-Report.ps1`. Expected: The report lists each design with its criteria results, disposition, open conditions, and any past-due exception, and matches the Design Review Records.

Test 8 — Exceptions are tracked

Do: Run `GWVEND-Validation.ps1` (exception section). Expected: Every granted exception is listed with its close date; any exception past its close date is flagged as standing lock-in debt.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-5 pass on the pilot design; the design is dispositioned and any conditions tracked
Expand pass	Tests 1-3, 6-8 pass across all partners; every build-status design is approved, no un-approved vendor object exists, and no exception is past due

6. Troubleshooting Guide — Vendor Technical Governance & Anti-Lock-In Playbook

Use `GWVEND-Discovery.ps1` for the register and source-environment recon, `GWVEND-Validation.ps1` for conformance and disposition state, `GWVEND-Report.ps1` for the finding report, and `GWVEND-Remediation.ps1` (preview first) to record remediation actions. This guide focuses on the core challenge: **handling non-conformant designs** without either rubber-stamping them or stalling the program.

A. A vendor design fails one or more anti-lock-in criteria

This is the normal case, not an error — the method is working.

Situation	Handling
Fails **portability** (proprietary-only pattern, no alternative)	Require the sanctioned/standard pattern, or a documented, accepted reason with an exit path. Disposition **Approved-with-conditions** (condition = adopt standard pattern or record accepted trade) or **Rejected**.
Fails **data ownership** (vendor-gated export, undocumented schema)	Require documented schema and on-demand full extract before build. Do not approve until the enterprise can get its data out unaided.
Fails **standards conformance** (proprietary where a standard exists)	Require the open standard (SAML/OIDC/SCIM, documented API, open format) or a recorded, accepted reason.
Fails **exit criteria** (no articulable exit)	Require the exit path, switching cost, and data-egress story at design time. "Deal with it later" is a fail.

B. A partner pushes back on the gate

Symptom	Cause	Remediation
"The gate is slowing us down"	Gate seen as optional review, not a build precondition	Re-confirm in the RACI that architectural disposition is enterprise-Accountable and is a precondition for build; escalate to the Program/Enterprise Owner if unresolved. The friction is intended — manage it through escalation, not by waiving the gate.
Vendor disputes a criterion result	Assumption behind the design was never validated	Return to the Vendor Assumption Log; validate or reject the assumption, then re-score. A disagreement usually traces to an unvalidated assumption treated as fact.
Design blocked, schedule at risk	No time-boxed exception path in use	Grant an owner-accountable, dated **exception** with a remediation commitment; record it and track it to closure (see D).

C. Escalation ladder for a rejected/blocked design

1. **Architect** ↔ **vendor** — resolve against the specific failing criterion (not the whole design).

2. **Program/Enterprise Owner arbitrates** — RACI-Accountable for escalation; decides proceed with conditions, exception, or rework.

1. **Exception (time-boxed)** — owner-accountable, dated, with a remediation commitment; recorded, never a silent waiver. Run `GWVEND-Report.ps1` to give the escalation the finding record it needs.

D. Exceptions piling up / going stale

• Every granted exception is standing lock-in debt until closed. Run `GWVEND-Validation.ps1` (exception section) on a cadence; any exception past its close date is flagged. Drive each to closure — an unclosed exception means a non-conformant design is now in production unremediated.

E. A vendor object exists with no approved design

Symptom	Cause	Remediation
`GWVEND-Discovery.ps1` finds a vendor-created enterprise app / group / config in the source environment that maps to no approved design	Work was built ahead of the gate	Flag it, trace it to a partner and a (missing) design, run it through the review method retroactively, and disposition it. Repeated occurrences mean the gate is being bypassed — escalate.

F. Register out of date / discovery returns little

Symptom	Cause	Remediation
Register missing in-flight designs	Not maintained on a multi-partner program	Re-run `GWVEND-Discovery.ps1` on a cadence and seed from current deliverables; make register intake step 1 of the runbook (§8).
Source-environment recon empty or errors	Graph SDK not installed / read scopes not consented / wrong tenant	`Install-Module Microsoft.Graph -Scope CurrentUser`; re-run and accept `Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`; confirm the correct source tenant.

G. Diagnostics quick reference

```
# Enumerate the register and source-environment vendor objects (read-only)
powershell -ExecutionPolicy Bypass -File .\GWVEND-Discovery.ps1 -RegisterPath .\vendor-design-register.csv

# Score conformance + disposition + exceptions
powershell -ExecutionPolicy Bypass -File .\GWVEND-Validation.ps1 -RegisterPath .\vendor-design-register.cs

# Finding report for the escalation
powershell -ExecutionPolicy Bypass -File .\GWVEND-Report.ps1 -RegisterPath .\vendor-design-register.csv
```

H. When to escalate to program leadership

If a partner repeatedly builds ahead of the gate, disputes the anti-lock-in bar itself (rather than a single criterion result), or an exception cannot be closed because the lock-in is now irreversible, treat it as a program-governance issue: raise it to the Program/Enterprise Owner (RACI-Accountable for escalation) with the finding report, and decide whether the design must be reworked, contractually addressed, or accepted as a recorded, owned trade.

7. Best Practices — Vendor Technical Governance & Anti-Lock-In Playbook

• Hold architectural authority; delegate execution. The single most important discipline: the enterprise decides *what the architecture is and whether a design conforms*; the vendor decides *how to build it*. Never let disposition authority drift into the "produce a design" or "build" rows of the RACI.

• Gate every design, without exception. A design-authority gate only works if it is mandatory. One design waved through on schedule pressure teaches every partner the gate is optional.

• Apply the same four criteria every time. Portability, data ownership, standards conformance, exit

criteria — a fixed bar applied consistently is what prevents lock-in accumulating. Re-arguing the bar per design is the same as having no bar.

• Treat exit criteria as a first-class requirement, not an afterthought. "We'll figure out exit later" is a fail. A design with no articulable exit is lock-in by definition; require the exit path, switching cost, and data-egress story at design time.

• Validate assumptions before they harden. An unchallenged vendor assumption silently becomes a design constraint. Log every assumption, mark it validated / unvalidated / rejected, and never let an unvalidated one pass into build.

• Prefer standards over proprietary shortcuts. Where an open standard exists (SAML/OIDC/SCIM, documented APIs, open data formats), require it — or require a documented, accepted reason for the proprietary path. Undocumented proprietary choices are how estates become un-leaveable.

• Make lock-in a design-time decision, not an accident. Some coupling is a legitimate, accepted trade. The method's job is to make that trade *explicit and owned* — recorded, reasoned, and signed off — never stumbled into.

• Keep the register current. New designs arrive faster than memory tracks on a multi-partner program. Re-run discovery on a cadence so nothing is built ahead of the gate unseen.

• Escalate cleanly; grant exceptions explicitly. A rejected design goes up the escalation ladder, not around the gate. A time-boxed exception is recorded, dated, owner-accountable, and tracked to closure — never a silent waiver.

• Be a good client. A clear, consistent, well-communicated bar makes partners better, not adversarial. Vendors deliver better work when they know exactly what "good" is up front.

• Document dispositions for audit and repeatability. Record every design's criteria results, disposition, conditions, and exceptions so the estate's architecture and its lock-in posture can be explained at any time.

8. Deliverables Summary — Vendor Technical Governance & Anti-Lock-In Playbook

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	The operating model between the enterprise architect and partners, where architectural authority sits, and the artifacts the method names and reuses
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	The vendor-design review method, the Anti-Lock-In Criteria checklist, the Governance RACI, and the escalation path — in ordered steps
5. Testing Guide	Governance and conformance gates with pass/fail thresholds
6. Troubleshooting Guide	Handling non-conformant designs and field issues
7. Best Practices	Field-proven recommendations for durable partner governance

PowerShell scripts

Script	What it does
`GWVEND-Discovery.ps1`	Inventories the vendor design register and cross-checks it against vendor-created objects readable in the source Microsoft 365 / Entra environment (read-only)
`GWVEND-Validation.ps1`	Scores each design against the four anti-lock-in criteria, confirms every entry is dispositioned, and flags un-approved objects and past-due exceptions (read-only)
`GWVEND-Report.ps1`	Review/finding report — per design: criteria results, disposition, open conditions, exceptions (read-only)
`GWVEND-Remediation.ps1`	Records remediation actions for non-conformant designs (disposition/condition updates); `-WhatIf` by default
`GWVEND-Set-ScopeTargeting.ps1`	Builds and previews the design-authority reviewer and pilot-scope groups

The Vendor-Governance Runbook (outline)

A one-page repeatable procedure the enterprise runs for every incoming vendor design, so the method is executed the same way regardless of partner or architect:

1. **Intake** — log the design in the register (`GWVEND-Discovery.ps1`); assign `DesignId`, owner, type, status = proposed.
1. **Assumptions** — extract dependencies into the Vendor Assumption Log; mark each validated / unvalidated / rejected.
1. **Baseline review** — compare the design to the Target Architecture Baseline (identity, integration, data ownership, standards).
1. **Anti-lock-in scoring** — score all four criteria (`GWVEND-Validation.ps1`): portability, data

ownership, standards conformance, exit criteria.

1. **Challenge** — require the sanctioned/standard pattern or a documented, accepted reason for any proprietary shortcut; require every assumption validated.

1. **Disposition** — record Approved / Approved-with-conditions / Rejected in the Design Review Record, with conditions; only Approved (or conditions-met) proceeds to build.

1. **Escalate if blocked** — run the escalation ladder (§6); grant and track a time-boxed exception only with owner accountability.

1. **Report & close** — `GWVEND-Report.ps1`; track conditions and exceptions to closure; re-run discovery on cadence.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Vendor execution that aligns to the target architecture — assumptions validated, every design reviewed against a fixed anti-lock-in bar and dispositioned by the enterprise, non-conformant designs escalated rather than waved through — with architectural authority firmly held by the enterprise and no long-term lock-in accumulating unseen.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWVEND-Discovery.ps1

Purpose: Read-only inventory for vendor technical governance. It (1) enumerates the vendor design register (a CSV/JSON of every in-flight vendor deliverable/design with owner, type, status, and the four anti-lock-in fields) and (2) optionally cross-checks it against vendor-created objects readable in the source Microsoft 365 / Entra environment — enterprise apps and groups — so designs that were built ahead of the design-authority gate surface. This is the governance baseline. Provide the register path as 'your-register.csv' (no angle brackets). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWVEND-Discovery.ps1 -RegisterPath .\vendor-design-register.csv Add -CheckSourceEnvironment to also read vendor-created objects in the source tenant. Source recon uses the Microsoft Graph PowerShell SDK and consents to Application.Read.All, Directory.Read.All (read-only). Install once: Install-Module Microsoft.Graph -Scope CurrentUser. EXPECTED RESULTS A listing of every register entry with owner, type, status, and disposition, plus a count of entries lacking a disposition; with -CheckSourceEnvironment, a list of vendor-created objects and any that map to no approved design (flagged for review). No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [switch]$CheckSourceEnvironment, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWVEND-Discovery.ps1 -RegisterPath .\vendor-design-register.csv Add -CheckSourceEnvironment to also read vendor-created objects in the source tenant. Source recon uses the Microsoft Graph PowerShell SDK and consents to Application.Read.All, Directory.Read.All (read-only). Install once: Install-Module Microsoft.Graph -Scope CurrentUser. EXPECTED RESULTS A listing of every register entry with owner, type, status, and disposition, plus a count of entries lacking a disposition; with -CheckSourceEnvironment, a list of vendor-created objects and any that map to no approved design (flagged for review). No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [switch]$CheckSourceEnvironment, [string]$CsvPath)

GWVEND-Remediation.ps1

Purpose: Applies a reviewed governance decision to a design in the vendor design register: set the disposition (Approved / Approved-with-conditions / Rejected), attach a condition, or record a time-boxed exception with a close date. It is intended to be run after a design has been reviewed against the anti-lock-in criteria and escalated per the runbook — it updates the register of record, it does not change any live system. Defaults to -WhatIf, so a plain run only PREVIEWS the register change and writes nothing. Provide the register path as 'your-register.csv' and the design id as 'GWVEND-001' (no angle brackets). HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GWVEND-Remediation.ps1 -RegisterPath .\vendor-design-register.csv -DesignId GWVEND-001 -Disposition "Approved-with-conditions" -Condition "Adopt SCIM provisioning; document exit path" Apply after review: powershell -ExecutionPolicy Bypass -File .\GWVEND-Remediation.ps1 -RegisterPath .\vendor-design-register.csv -DesignId GWVEND-001 -Disposition "Approved-with-conditions" -Condition "Adopt SCIM provisioning; document exit path" -WhatIf:$false Optional: -ExceptionCloseDate 2026-09-30 to record a time-boxed exception. EXPECTED RESULTS In preview: the exact register fields that WOULD

change for the design. When applied: the register file is updated in place (a .bak backup is written first). Re-run GWVEND-Validation.ps1 to confirm the design is now dispositioned/conformant. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$RegisterPath, [Parameter(Mandatory)] [string]$DesignId, [ValidateSet('Approved','Approved-with-conditions','Rejected')] [string]$Disposition, [string]$Condition, [string]$ExceptionCloseDate)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GWVEND-Remediation.ps1 -RegisterPath .\vendor-design-register.csv -DesignId GWVEND-001 -Disposition "Approved-with-conditions" -Condition "Adopt SCIM provisioning; document exit path" Apply after review: powershell -ExecutionPolicy Bypass -File .\GWVEND-Remediation.ps1 -RegisterPath .\vendor-design-register.csv -DesignId GWVEND-001 -Disposition "Approved-with-conditions" -Condition "Adopt SCIM provisioning; document exit path" -WhatIf:$false Optional: -ExceptionCloseDate 2026-09-30 to record a time-boxed exception. EXPECTED RESULTS In preview: the exact register fields that WOULD change for the design. When applied: the register file is updated in place (a .bak backup is written first). Re-run GWVEND-Validation.ps1 to confirm the design is now dispositioned/conformant. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$RegisterPath, [Parameter(Mandatory)] [string]$DesignId, [ValidateSet('Approved','Approved-with-conditions','Rejected')] [string]$Disposition, [string]$Condition, [string]$ExceptionCloseDate)

GWVEND-Report.ps1

Purpose: Read-only review report. Per design it presents the four anti-lock-in criteria results (portability, data ownership, standards conformance, exit criteria), the disposition (Approved / Approved-with-conditions / Rejected / none), any open conditions, and any exception with its close date. It ranks designs by review risk (failed or unscored criteria, no disposition, past-due exception) so the highest-attention designs surface first — the record an escalation needs. Provide the register path as 'your-register.csv' (no angle brackets). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWVEND-Report.ps1 -RegisterPath .\vendor-design-register.csv Optional: -CsvPath .\gwvend-findings.csv to export the finding rows. No live-system access is required; the register file is the input. EXPECTED RESULTS A ranked finding table (highest review risk first) with criteria results, disposition, open conditions, and exceptions. No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWVEND-Report.ps1 -RegisterPath .\vendor-design-register.csv Optional: -CsvPath .\gwvend-findings.csv to export the finding rows. No live-system access is required; the register file is the input. EXPECTED RESULTS A ranked finding table (highest review risk first) with criteria results, disposition, open conditions, and exceptions. No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [string]$CsvPath)

GWVEND-Set-ScopeTargeting.ps1

Purpose: Creates (or previews) the two security groups this governance model targets: the design-authority reviewer group "Vendor Governance Design Authority - Tenant - Steady" (the enterprise

architects who disposition vendor designs at the gate) and the pilot scope "Vendor Governance Pilot Scope - Pilot - Pilot" (the first workstream the gate is applied to). Use the reviewer group as the named architectural authority in the RACI and the pilot scope to stage the rollout. By default it PREVIEWS (no changes); pass -Create to create groups that do not yet exist. These are placeholder names — rename to your convention. HOW TO RUN Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\GWVEND-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\GWVEND-Set-ScopeTargeting.ps1 -Create Uses the Microsoft Graph PowerShell SDK; consents to Group.ReadWrite.All, Directory.Read.All (read-only in preview mode). Install once: Install-Module Microsoft.Graph -Scope CurrentUser. EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. No changes are made in preview mode. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([switch]$Create)

How to run: Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\GWVEND-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\GWVEND-Set-ScopeTargeting.ps1 -Create Uses the Microsoft Graph PowerShell SDK; consents to Group.ReadWrite.All, Directory.Read.All (read-only in preview mode). Install once: Install-Module Microsoft.Graph -Scope CurrentUser. EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. No changes are made in preview mode. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([switch]$Create)

GWVEND-Validation.ps1

Purpose: Read-only conformance check over the vendor design register. It (1) scores every design on the four anti-lock-in criteria — portability, data ownership, standards conformance, exit criteria — from the register's Pass/Fail/Unknown fields; (2) confirms every entry has a disposition and that no build-status design is missing a passed criterion; and (3) flags any exception that is past its close date. Provide the register path as 'your-register.csv' (no angle brackets). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWVEND-Validation.ps1 -RegisterPath .\vendor-design-register.csv Optional: -DesignId GWVEND-001 to score a single design. No live-system access is required for scoring; the register file is the input. EXPECTED RESULTS PASS/FAIL lines per design against the four criteria, a list of undispositioned or non-conformant build-status designs, and any past-due exception. No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [string]$DesignId)

How to run: powershell -ExecutionPolicy Bypass -File .\GWVEND-Validation.ps1 -RegisterPath .\vendor-design-register.csv Optional: -DesignId GWVEND-001 to score a single design. No live-system access is required for scoring; the register file is the input. EXPECTED RESULTS PASS/FAIL lines per design against the four criteria, a list of undispositioned or non-conformant build-status designs, and any past-due exception. No changes are made. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$RegisterPath, [string]$DesignId)